

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SESSION 4

Coalesced memory access and device (GPU) memory types

Groothuis Thomas (r0955783)
De Smet Feniks (r0891378)

Academic year 2025–2026

Contents

1 Problem & Goals **2**
- 1.1 Memory Coalesence . . . 2
- 1.2 GPU memory types . . . 2

2 Methodology **3**
- 2.1 Memory Coalesence . . . 3
  - 2.1.1 Copy Kernel . . . 3
  - 2.1.2 Transpose Kernel . . . 4
- 2.2 Memory types . . . 4
  - 2.2.1 Global Memory kernel . . . 4
  - 2.2.2 Constant Memory kernel . . . 5
  - 2.2.3 Shared Memory kernel . . . 5

3 Results **7**
- 3.1 Memory Coalesence . . . 7
  - 3.1.1 Copy Kernel . . . 7
  - 3.1.2 Transpose kernel . . . 9
- 3.2 GPU memory types . . . 11

4 Conclusion **12**

A Code Implementations **12**
- A.1 Part 1: . . . 12
  - A.1.1 CUDA code . . . 12
  - A.1.2 Data Collection Script . . . 15
- A.2 Part 2: . . . 17
  - A.2.1 CUDA code . . . 17
  - A.2.2 Data Collection Scripts . . . 20

1 Problem & Goals

In this lab, we investigate the impact of memory architecture on the execution speed of GPU kernels. Because the performance of parallel algorithms depends heavily on efficient data delivery, understanding the underlying hardware model is essential. Physically, the architecture is divided into slow off-chip memory (GDDR6 VRAM) and high-speed on-chip memory. This on-chip memory includes thread-specific registers and a unified cache per Streaming Multiprocessor (SM). This unified cache contains the configurable shared memory, alongside an unpartitioned L1/texture pool.

To understand how to optimally utilize this hardware, this lab is divided into two parts:

1.1 Memory Coalesence

When threads request data from global memory, this data always passes through the shared L2 cache before reaching the L1 cache of the specific Streaming Multiprocessor. The efficiency of this data transfer is strictly determined by the memory access patterns of the threads within a single warp (a group of 32 parallel threads). Crucially, the memory controller always fetches data from global memory in fixed-size blocks (128-byte cache lines) regardless of how much data a thread actually needs. When the threads in a warp execute a memory instruction, the hardware evaluates the requested addresses. If the threads access contiguous and correctly aligned memory locations, the data for all 32 threads fits neatly into a single 128-byte cache line. The hardware can satisfy the entire warp with just one memory transaction. This highly optimized utilization of a fetched cache line is known as memory coalescing. If the threads within the warp request are scattered or unaligned memory addresses, the hardware must still fetch full 128-byte cache lines for each distinct memory segment touched. Because only a tiny fraction of each fetched cache line is actually used by the threads, a massive amount of memory bandwidth is wasted transferring unneeded data. This forces the hardware to issue multiple separate memory transactions just to satisfy one warp instruction, leading to bandwidth saturation and a severe bottleneck known as uncoalesced memory access.

The primary objective of this section is to implement and analyse both a copy kernel and a transpose kernel operating on a square $N \times N$ matrix. Per the assignment specifications, the matrices are stored as 1D float arrays in row-major order. By measuring the execution time and computing the effective bandwidth for each kernel, we can directly quantify the hardware penalty caused by uncoalesced memory accesses.

1.2 GPU memory types

Regarding the logical memory spaces explicitly accessible to a CUDA programmer, the architecture provides global, constant, shared, and texture memory. Texture memory physically resides within the off-chip GDDR6 VRAM alongside global memory, but it is strictly read-only and utilizes dedicated Texture Mapping Units (TMUs) on the SMs for hardware-accelerated data filtering and interpolation. For the scope of this report, texture memory will not be discussed further.

Both global and constant memory are physically located off-chip in the VRAM. Global memory relies on the memory bus and the L2/L1 caches to deliver data to the threads. If all threads within a warp request the exact same memory address, the L1 cache can perform a uniform read, fetching the data in a single transaction and broadcasting it to the entire warp. While the L1 cache is highly optimized to serve standard memory fetches for multiple threads simultaneously (such as during a coalesced access), it cannot perform a uniform broadcast and a standard memory fetch at the exact same time. Relying solely on the L1 cache for both types of operations forces the hardware to execute them sequentially, creating a bottleneck that limits overall parallel throughput.

To alleviate this, the architecture provides constant memory. Constant memory is a read-only space initialized by the CPU. When a warp requests constant data, it is routed through a dedicated datapath to a specialized constant cache located directly on the SM. Because this cache operates independently of the L1 cache, it can efficiently broadcast its data to a warp at the exact same time the L1 cache is fetching other global data. This parallel operation effectively increases the total data throughput available to the SM.

Finally, shared memory is a fast, on-chip cache controlled directly by the programmer. Its main practical use is bypassing global memory bottlenecks. If an algorithm requires threads to read data in a scattered or unaligned way, doing so directly from global memory will ruin your bandwidth. Instead, we use shared memory as a temporary workspace. The workflow is straightforward: first, threads work together to load data from global memory in clean coalesced blocks and drop it into shared memory. Once the data is safely on-chip, threads

can read it in whatever messy or complex pattern they need without a severe performance penalty. After the calculations are done, the threads write the final results back out to global memory, once again using fast, coalesced transactions.

To evaluate the practical impact of these memory types, we will compare three different matrix multiplication kernels: a baseline kernel relying entirely on global memory, a tiled kernel using shared memory, and a kernel utilizing constant memory. By measuring the execution time of each implementation, we can directly quantify their performance differences.

2 Methodology

Just like in previous labs, we use cudaEvent timers immediately before and after our kernel launches to accurately measure execution time. For both parts of this lab, we configure our execution grid using a fixed tile size, defining TILE_DIM as 32. This means our thread blocks are always set to a 2D structure of 32 by 32 threads, implemented in the code as dim3 block(TILE_DIM, TILE_DIM). To determine exactly how many of these blocks are needed to cover the entire $N \times N$ matrix, we calculate the grid dimensions dynamically using ceiling division: dim3 grid((N + TILE_DIM - 1) / TILE_DIM, (N + TILE_DIM - 1) / TILE_DIM). By adding TILE_DIM - 1 to the matrix dimension N before dividing by TILE_DIM, we force the standard C++ integer division to always round up. This is a crucial safeguard: it guarantees that we launch enough thread blocks to process every single element of the matrix, even if N is not a perfect multiple of 32. Consequently, this ceiling division is also the exact reason why we must perform a bounds check for every thread. If N is not an exact multiple of 32, the grid will launch surplus threads. By validating their global coordinates (e.g., if ($x < N$ && $y < N$), we ensure that these out-of-bounds threads safely bypass the core logic and perform no work, preventing illegal memory accesses.

2.1 Memory Coalesence

To evaluate and compare the performance of the kernels, we rely on two primary metrics: execution time and effective global memory bandwidth. The execution time allows us to directly observe how long each kernel takes to complete its task, while the effective global memory bandwidth demonstrates the overall throughput. In all our implementations, every thread is mapped directly to a single matrix element. Furthermore, although it was not strictly required for this section, we also developed a shared memory version of each kernel to serve as an additional test case.

When calculating the effective bandwidth, we are actually measuring the theoretical amount of data that must be moved over the memory bus. We calculate this by taking the total required data for a read and a write (2 * N * N elements) and dividing it by the total execution time of the kernel. Internally, kernels will often move more data because of how they operate, such as routing data through the SM's shared memory in our extra test cases. This internal movement of data is entirely based on the kernel code. Thus, by keeping the data size constant in our formula, we accurately measure the efficiency of a kernel's specific strategy in moving the required payload over the memory bus.

2.1.1 Copy Kernel

The copy kernel takes in a matrix and copies it. This is done by copying the array of the original matrix into another array.

The global copy kernel begins with each thread computing its global x and y indices ($blockIdx.x * blockDim.x + threadIdx.x$ and correspondingly for y) to identify its target element. After passing the boundary check, the thread performs a direct one-to-one copy operation. It reads the element located at the 1D index $y * N + x$ in the input array and writes it to the corresponding index $y * N + x$ in the output array.

While the global copy kernel transfers data directly, the shared copy kernel introduces an intermediate staging step using a small 2D array allocated in the SM's shared memory, referred to as a tile. First, each thread determines the specific element it is responsible for copying by calculating its global coordinates via the formula int x = blockIdx.x * TILE_DIM + threadIdx.x (with the identical logic applied for y). The element is then transferred from the global index y * N + x in the input array to the local coordinates [threadIdx.y][threadIdx.x] within the tile. After this loading phase, all threads within the block synchronize to ensure the tile is fully populated. Finally, the element is read from [threadIdx.y][threadIdx.x] in the tile and written back directly to

the index y * N + x in the global output array. Both of these memory operations are strictly protected by a bounds check to prevent accessing data outside the matrix.

2.1.2 Transpose Kernel

The transpose kernel takes a matrix and transposes it. This means reading a row and writing to a column.

In global transpose kernel we first determine the global indices akin to how we did so for the global copy kernel. We do the bounds check but then we move the element from $y * N + x$ in the input array to $x * N + y$ in the output array.

In the shared transpose kernel, the shared memory tile dimensions are defined as [TILE_DIM][TILE_DIM + 1]. This extra column of padding is added to prevent shared memory bank conflicts. During a transpose, threads read data row-by-row but must write it back column-by-column. Since shared memory is divided into 32 banks, reading a column from a standard 32×32 tile would cause all threads to hit the same bank simultaneously. The padding shifts the memory stride, ensuring that elements in a column are distributed across different banks for maximum throughput.Beyond this local tile optimization, we also manually recompute the block coordinates to mitigate partition camping:

$$blockIdx_x = (blockIdx.x + blockIdx.y) \bmod gridDim.x$$

$$blockIdx_y = blockIdx.x$$

While the global transpose kernel is primarily bottlenecked by uncoalesced memory accesses,making hardware-level tuning redundant. The shared transpose kernel is fast enough to be limited by DRAM partition congestion. This diagonal mapping ensures an even distribution of write operations across memory channels. Although the hardware specifics of DRAM partitions are outside the scope of this lab, we will use these recalculated values as the effective coordinates for the remainder of this analysis. With the coordinates and tile optimized, the kernel follows a similar workflow to the copyShared kernel, but with a critical distinction in the access pattern. Data is first loaded from the global input array at index $y \cdot N + x$ into the tile at $[threadIdx.y][threadIdx.x]$. After a synchronization barrier, the actual transposition is achieved by reading from the tile at the swapped indices $[threadIdx.x][threadIdx.y]$ and storing the element into the global output array at index $y \cdot N + x$. This approach allows the non-linear movement to happen within the high-speed shared memory, while keeping all reads and writes to the global memory perfectly coalesced.

2.2 Memory types

To evaluate the performance impact of different memory hierarchies, we compare the kernel execution times of three distinct implementations: one strictly utilizing global memory, one leveraging shared memory, and one utilizing constant memory. The fundamental operation of multiplying matrix A by matrix B consists of taking the dot product of the r-th row of matrix A and the c-th column of matrix B to compute the corresponding element at coordinates (r, c) in the resulting matrix C. Mathematically, for the $N \cdot N$ matrices, this is expressed as:

$$C_{r,c} = \sum_{i=0}^{N-1} A_{r,i} \cdot B_{i,c}$$

2.2.1 Global Memory kernel

First, each thread determines its target column (c) and row (r) coordinates in the output matrix C using the standard global thread index formulas:

$$c = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

$$r = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y}$$

Following a boundary check to ensure these coordinates fall within the actual matrix dimensions, the thread computes the dot product for its assigned element. A local accumulator variable is initialized to zero. The thread then iterates through the inner matrix dimension k (from 0 to $N - 1$), multiplying the corresponding elements from row r in matrix A and column c in matrix B. Because the 2D matrices are flattened into 1D

arrays in memory, these elements are accessed using the indices $A[r \cdot N + k]$ and $B[k \cdot N + c]$. After the loop finishes accumulating the products, the final sum is written to the global output array at the corresponding flattened index: $C[r \cdot N + c]$.

2.2.2 Constant Memory kernel

The constant memory kernel implements the same core matrix multiplication logic as the global memory version, with the key distinction that matrix A is stored in the GDDR6's constant memory. To utilize this memory space, the array must be declared globally using the __constant__ keyword, making it globally accessible to all threads during execution. A strict hardware limitation of constant memory is its capacity constraint of exactly 64 KB. Assuming standard 32-bit floating-point numbers (4 bytes per element), this restricts the maximum storable matrix size to 128×128 elements ($128 \cdot 128 \cdot 4$ *bytes* $=65,536$ *bytes*). Consequently, we established this as the maximum supported matrix dimension for this specific kernel implementation. Prior to kernel execution, the constant memory array is populated with the host data using the cudaMemcpyToSymbol function. The reason we use the A array inside of the constant memory is because when you look at the way the matrix multiplications happen for multiples of 32 in a row of a matrix. C will also have 32 elements in the same row. This means that every multiplication performed by the threads in that row will always use the same element from matrix A. Therefore, this value can be streamed to all threads simultaneously, which is exactly what constant memory is optimized for. This then allows us to stream the A element while pulling the B elements through the caches and memory bus.

2.2.3 Shared Memory kernel

This kernel employs an optimization technique known as tiling. This approach logically divides the large global matrices into smaller, more manageable sub-matrices, or 'tiles', of size TILE_DIM $\times$ TILE_DIM. To understand this process, it is easiest to look at the output matrix C. This matrix is partitioned into a grid of tiles and each thread is responsible for calculating a single element within one of these tiles. To compute a specific tile of C efficiently, the input matrices A and B are similarly divided into tiles. The calculation then proceeds in a series of stages. During each stage, the kernel loads one tile from matrix A and a corresponding tile from matrix B into shared memory arrays. Once these smaller tiles are loaded, the threads perform a matrix multiplication on them, calculating and accumulating the partial sums for their assigned C elements. After a stage is completed, the kernel advances to the next set of tiles along the respective row of A and column of B. This staged process of loading and accumulating repeats until all necessary input tiles have been processed, resulting in the fully calculated tile of matrix C.

For an individual thread, this process translates into the following sequence of operations. First, the thread determines the global matrix coordinates of the element it is responsible for calculating:

$$r = \text{blockIdx.y} \cdot \text{TILE_DIM} + \text{threadIdx.y}$$

$$c = \text{blockIdx.x} \cdot \text{TILE_DIM} + \text{threadIdx.x}$$

Having established its target coordinates in the output matrix C, the kernel must determine the total number of phases (or stages) required to compute the full dot product. For an $N \times N$ matrix processed by blocks of TILE_DIM $\times$ TILE_DIM threads, the exact number of phases is N/TILE_DIM. However, because matrix dimensions are not always perfect multiples of the tile dimension (which is typically set to 32 to align with the GPU hardware warp size), we must compute the ceiling of this division to ensure all elements are covered. In integer arithmetic, this ceiling calculation is achieved using the following formula:

$$(N + \text{TILE_DIM} - 1)/\text{TILE_DIM}$$

This value serves as the upper bound for the main computational loop. We use the loop variable m to denote the current phase. During each phase m, the threads collaboratively load the required elements from the global matrices into the shared memory tiles for A and B, preparing the data for the localized dot product computation.

When populating the shared memory tile tA, each thread evaluates the condition $r < N$ && $(m \cdot \text{TILE_DIM} + \texttt{threadIdx.x}) < N$. The first part of this condition verifies that the assigned global row exists within the matrix bounds. The second part checks whether the specific element being fetched from that row exists, ensuring the sliding tile has not exceeded the rightmost boundary of matrix A. If both conditions are met, the thread copies the element from global memory at index $r \cdot N + m \cdot \text{TILE_DIM} + \texttt{threadIdx.x}$ into the shared

memory tile at `tA[threadIdx.y][threadIdx.x]`. If the bounds are exceeded, the shared memory location is padded with 0 to prevent undefined behavior during calculation.

Similarly, for the shared memory tile tB, the thread evaluates the condition $(m \cdot \text{TILE_DIM} + \texttt{threadIdx.y}) < N \;\&\&\; c < N$. Here, the second part verifies that the assigned global column exists, while the first part ensures that the specific element being fetched from that column has not exceeded the bottom boundary of matrix B. If valid, the thread copies the element from global memory at index $(m \cdot \text{TILE_DIM} + \texttt{threadIdx.y}) \cdot N + c$ into `tB[threadIdx.y][threadIdx.x]`. Again, if the check fails, the location is padded with 0.

Once the data for the current phase has been loaded, all threads within the block are synchronized. This guarantees that the shared matrices tA and tB are fully populated before any calculations begin. Next, each thread iterates through the inner dimension of the current tile, with the index k running from 0 to TILE_DIM$-$1. In this loop, the thread multiplies the element from row threadIdx.y of tA with the corresponding element from column threadIdx.x of tB. Mathematically, these products are accumulated into a local variable s using the following summation:

$$s = s + \sum_{k=0}^{\text{TILE_DIM}-1} \text{tA}[\text{threadIdx.y}][k] \cdot \text{tB}[k][\text{threadIdx.x}]$$

After completing this calculation, a second thread synchronization is required. This crucial step prevents faster threads from advancing to the next phase and overwriting the shared memory tiles while slower threads are still finishing their current computations.Finally, after all phases are complete, we perform a bounds check to verify whether the thread's target coordinates fall within the actual dimensions of the matrix. If the thread passes this check, the final accumulated sum s is written out to the global result matrix C at the linear index $r \cdot N + c$.

3 Results

3.1 Memory Coalesence

3.1.1 Copy Kernel

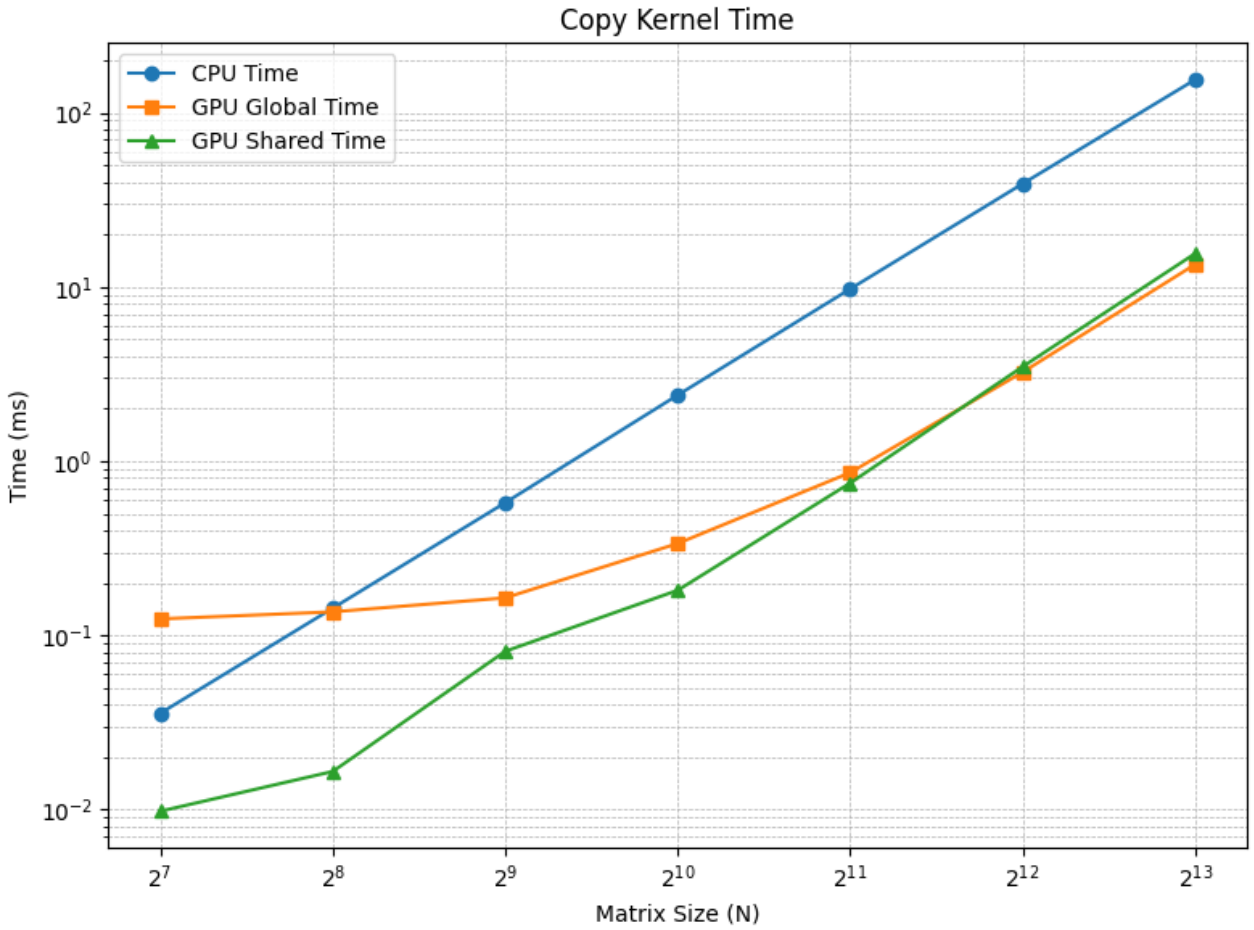


Figure 1: Execution time of copy kernel.

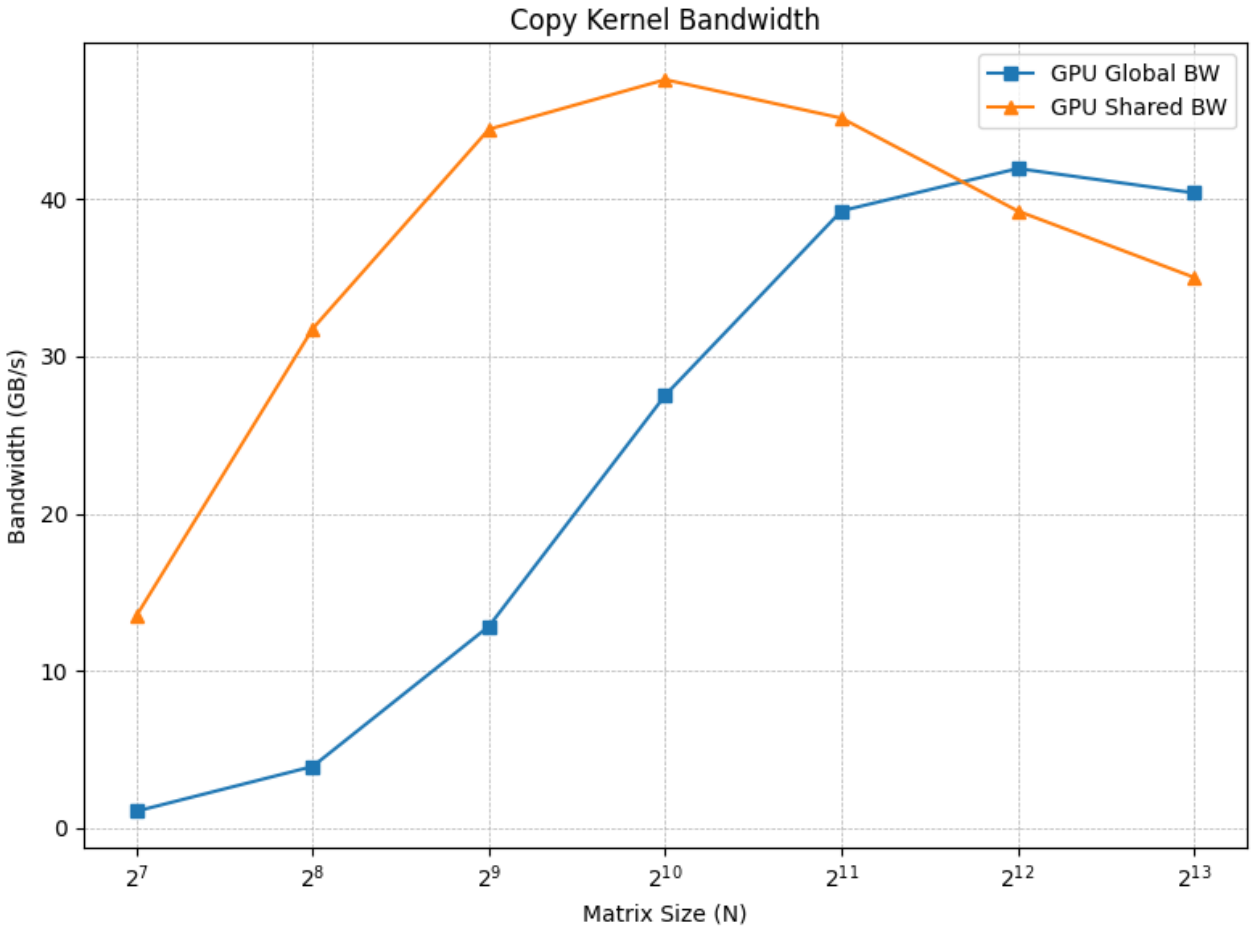


Figure 2: Bandwidth of copy kernel.

The memory access pattern of the copy kernel is inherently coalesced for both the global and shared memory implementations. Because the operation consists of a simple 1-to-1 transfer between arrays, the threads read and write the data in a perfectly linear sequence.

Figure 1 shows that for smaller matrix sizes, the CPU implementation is actually faster than the GPU global memory implementation. This behavior can be attributed to three main factors. First, for small matrices, the entire dataset fits easily into the large, ultra-fast L1 and L2 caches of the CPU, making memory retrieval nearly instantaneous. Second, the GPU suffers from a fixed kernel launch overhead that outweighs the computation time for small tasks. Finally, small matrices do not generate enough threads to fully occupy the GPU, leading to underutilization where the hardware cannot effectively hide the latency of global memory accesses.

Figure 2 illustrates the effective bandwidth. As the matrix size increases, the global memory implementation eventually overtakes the shared memory implementation, achieving both a lower execution time and a higher bandwidth. Because a simple copy operation involves absolutely no data reuse, utilizing shared memory introduces unnecessary instruction overhead (requiring an extra read and write per element) and forces idle time due to thread synchronization. Furthermore, allocating shared memory limits the overall occupancy of the Streaming Multiprocessors. Consequently, for a straightforward copy operation, the direct global memory approach is the most efficient method.

Although the theoretical maximum bandwidth of the GPU is 80 GB/s, the observed peak bandwidth for the global copy kernel stabilizes around 45 GB/s. This discrepancy is expected in GPU programming. The theoretical peak assumes 100% continuous bus utilization without any overhead. In reality, the achievable bandwidth is bounded by DRAM refresh cycles, addressing overhead, and pointer arithmetic instructions within the kernel. Furthermore, standard benchmarks typically only achieve 70% to 80% of the theoretical

maximum, making our empirically measured bandwidth consistent with practical expectations for a custom unoptimized kernel.

3.1.2 Transpose kernel

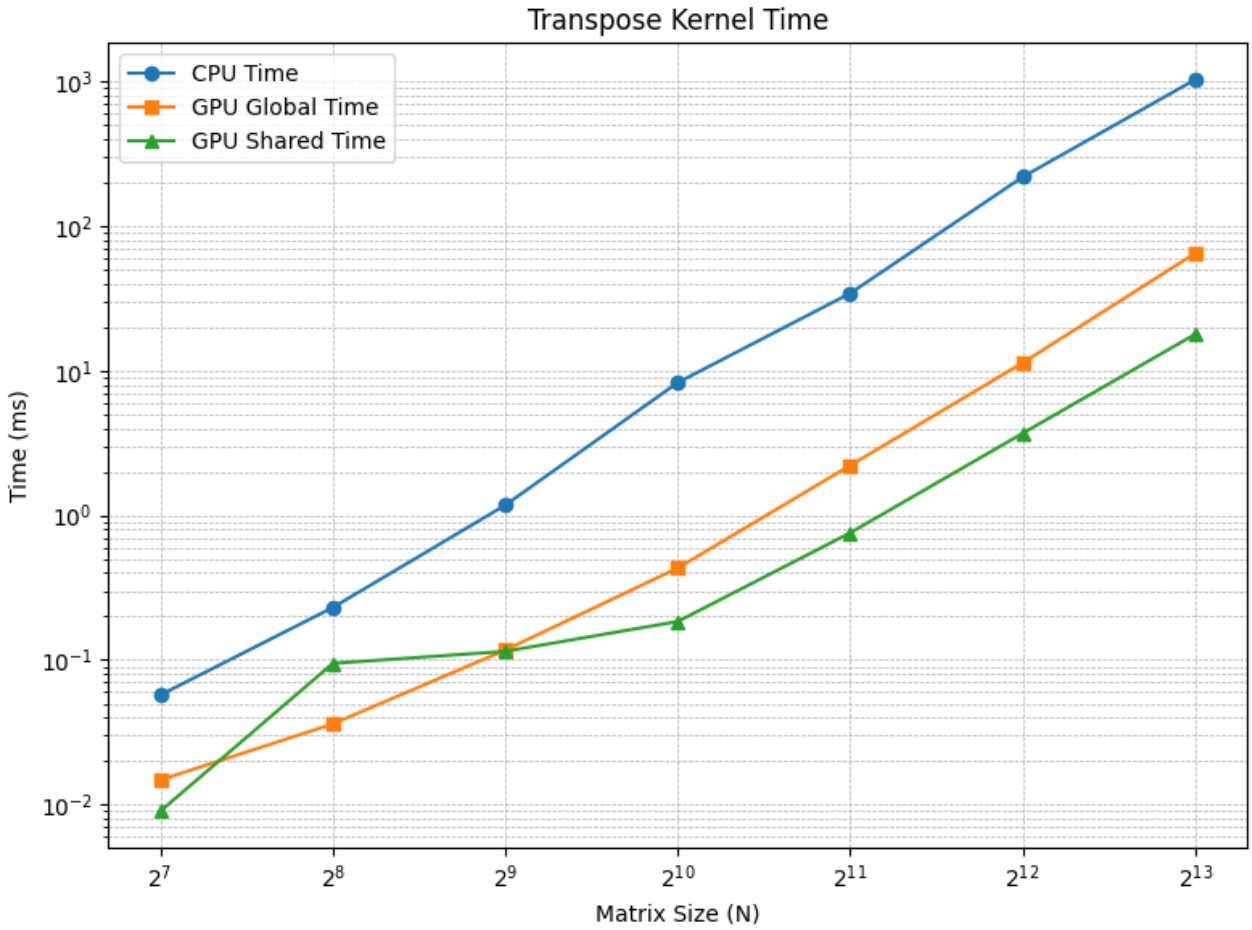


Figure 3: Execution time of transpose kernel.

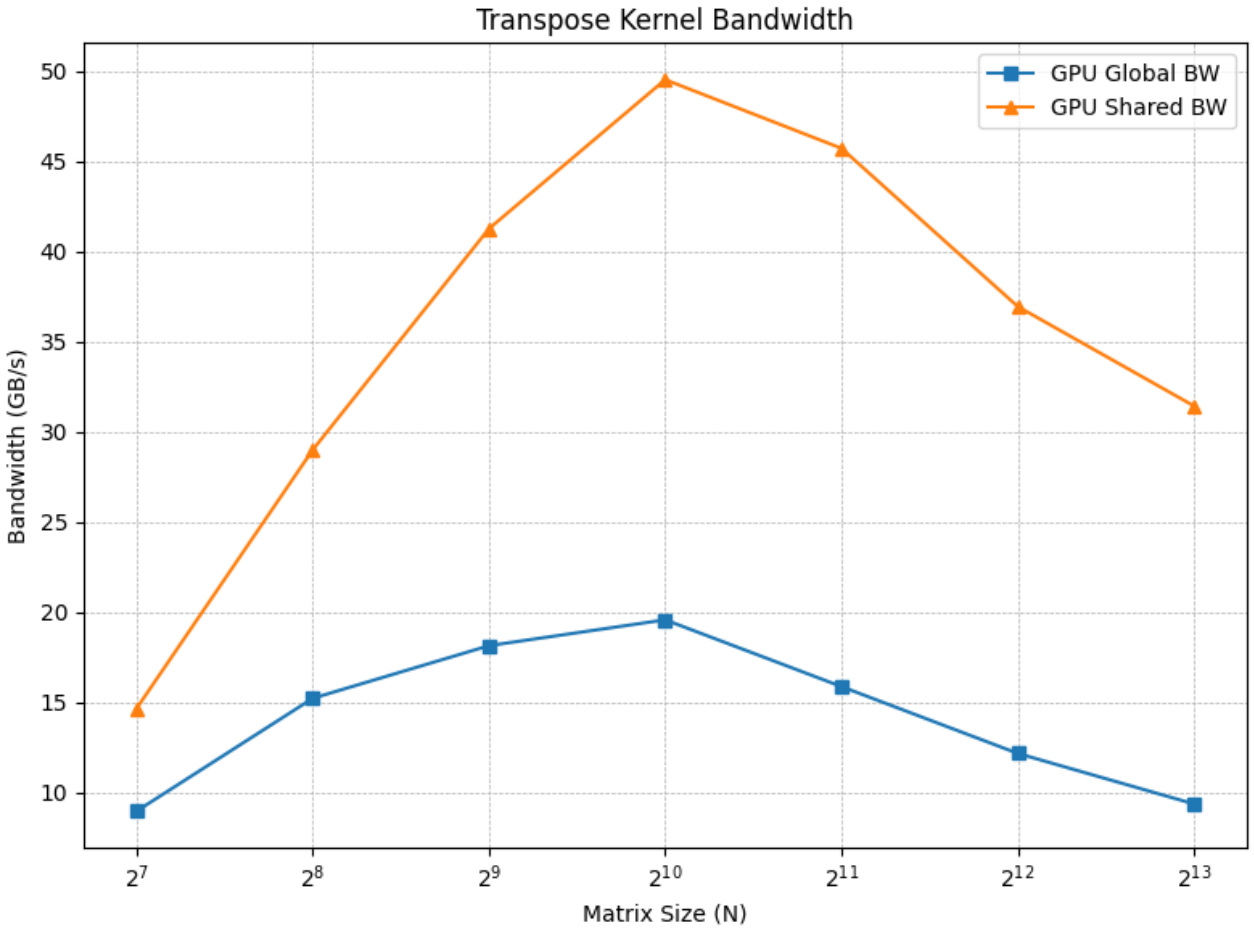


Figure 4: Bandwidth of transpose kernel.

In the global memory implementation of the transpose kernel, the memory reads are coalesced, but the writes are severely uncoalesced. When reading, the threads within a warp access contiguous memory segments. However, when writing the transposed elements, the threads are forced to write to a column, resulting in scattered, strided memory accesses. This fragments the memory transactions, saturates the memory bus with overhead, and significantly degrades the effective bandwidth, as illustrated in Figure 4.

To understand this performance drop mathematically, consider a single warp of 32 threads. Since each thread processes a 4-byte float, the warp needs to write a total of 128 bytes of useful data. Modern GPU memory controllers process transactions in minimum chunk sizes (sectors) of 32 bytes. If the memory access is fully coalesced, the 128 bytes fit perfectly into contiguous sectors, allowing the hardware to transport exactly 128 bytes. This results in 100% bandwidth efficiency. However, during an uncoalesced transpose write, each thread in the warp writes to a completely different memory row. Because these addresses are spaced far apart, the memory controller must issue a separate 32-byte transaction for every single thread. The hardware is therefore forced to physically transport 1024 bytes (32 threads * 32 bytes) just to store the 128 bytes of actual data. This means the hardware is performing 8 times more work than necessary, reducing the theoretical bandwidth efficiency to a mere 12.5%.

Figure 3 demonstrates that the shared memory implementation is vastly superior for large matrices, although it is temporarily slower at the smallest matrix sizes. This initial performance dip occurs because shared memory introduces overhead, making it temporarily slower for very small datasets. In contrast, when using global memory, the L2 cache can buffer the uncoalesced writes, thereby mitigating the VRAM latency penalty of global memory access. Consequently, the extra instructions required by the shared memory kernel such as computing tile indices, writing to shared memory, and explicitly synchronizing threads with __syncthreads() introduce an overhead that outweighs the architectural benefits at that scale. However, for larger matrices,

the shared memory implementation achieves significantly higher bandwidth and lower execution times. By utilizing tiling, the kernel performs the actual transposition locally within the fast shared memory. Threads read the data from global memory in a coalesced manner, transpose it in shared memory, and then write the transposed tile back to global memory, which is now also fully coalesced. This approach ensures that the slow global memory bus is strictly accessed using optimized patterns. The uncoalesced scattering is completely isolated to the shared memory, which is specifically designed to handle non-sequential accesses efficiently.

3.2 GPU memory types

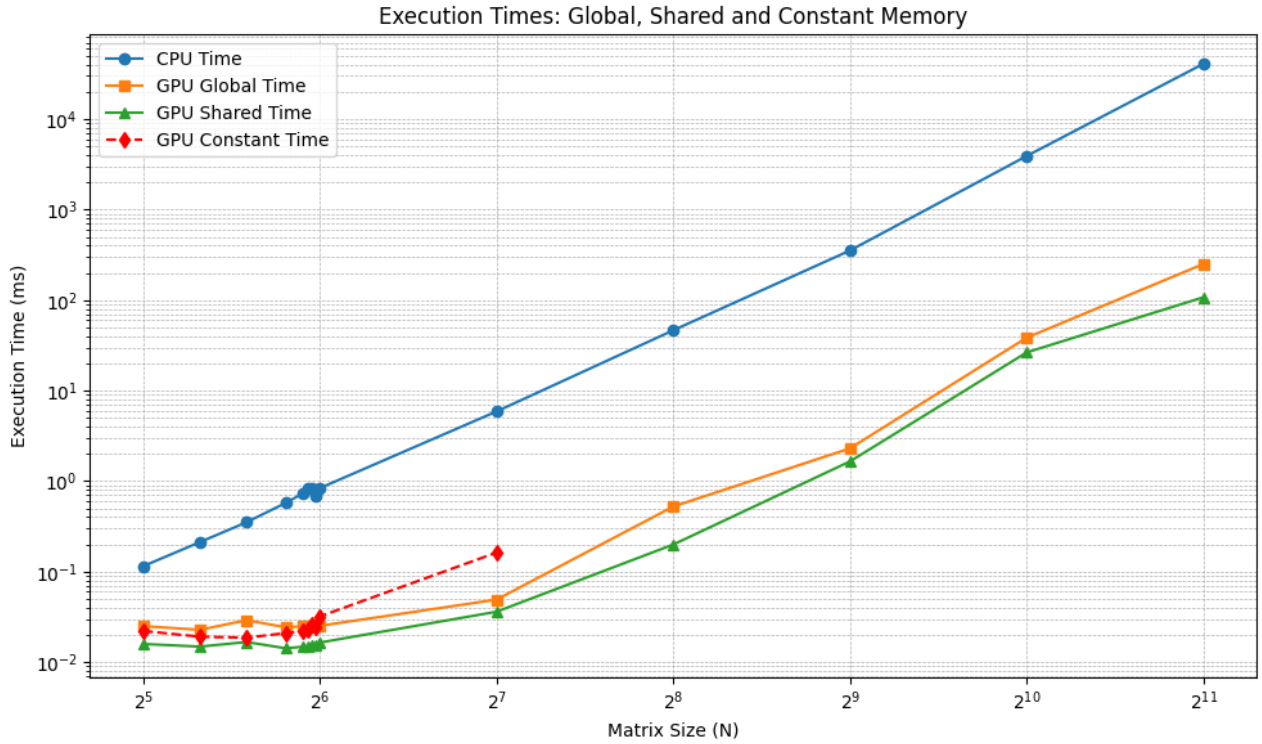


Figure 5: Execution time of kernels

Figure 5 shows the execution times of the different matrix multiplication kernels. As expected, the CPU implementation is by far the slowest, highlighting the necessity of GPU acceleration for this type of workload.

Among the GPU implementations, the kernel utilizing shared memory is clearly the fastest. This is because matrix multiplication is inherently a "cache"-bound operation, and tiling allows us to heavily cache the data in the fast shared memory. To illustrate this efficiency, consider a block size of 16×16. Without shared memory, each thread must fetch a value from global memory for every single calculation step. For a warp of 32 threads, this results in 32 accesses per step, totaling $32 \times 32 = 1024$ global memory accesses to compute their elements. By using tiling, the entire block collaboratively loads a tile into shared memory just once. Consequently, each thread only accesses the slow global memory 2 times per phase (once for matrix A, once for matrix B). For the same 32 threads, this reduces the total global accesses to just 64. The ratio of 1024/64 perfectly equals our block size of 16, representing a massive reduction in global memory traffic and explaining the dominant performance of the shared memory kernel.

When excluding the CPU, the global memory implementation is overall the slowest of the GPU kernels for larger matrices. However, it performs relatively well at smaller sizes as long as the data fits comfortably within the L1 and L2 caches. Assuming matrix dimensions are multiples of 32, the memory access pattern is actually quite favorable: all threads in a warp require the exact same value from a row in matrix A, while needing contiguous values from a column in matrix B. Modern GPUs optimize this by executing a uniform read for matrix A, where the L1 cache fetches the single required value and broadcasts it to all cores. This is followed by fetching the required cache lines for matrix B, achieving coalesced memory access. However, as the

matrix size grows, the required data massively exceeds the L1 cache capacity, causing the cache to overflow and performance to degrade steadily.

The behavior of the constant memory kernel is particularly interesting when compared to this global implementation. Constant memory relies on a 64 KB space in the GDDR6 DRAM, backed by a dedicated, very small 8 KB cache per Streaming Multiprocessor (SM). Because of this architecture, the constant cache can fetch and broadcast the required value from matrix A extremely fast, working in parallel with the unified L1 cache, which fetches matrix B in a coalesced manner. As seen in the graph, this hybrid approach, streaming data from two different memory subsystems simultaneously, initially outperforms the standard global memory kernel. However, as the matrix size increases, we begin to observe performance fluctuations and slowdowns even before $N = 64$. This early degradation occurs because the highly restrictive 8 KB constant cache quickly reaches its capacity. When cache misses occur, the constant memory is forced to fetch data directly from the much slower VRAM. Furthermore, because constant memory is strictly optimized for broadcasting a single value to all threads, any requests for different addresses cause the cache to severely serialize the memory requests. The most dramatic spike in execution time occurs further along the curve, driven by the L1 cache. While the constant cache is already suffering from serialization penalties for matrix A, the L1 cache (typically around 32 KB) is working to cache the columns of matrix B. As the workload scales past $N = 64$, this L1 cache also reaches saturation. The simultaneous cache misses from both the depleted 8 KB constant cache and the overwhelmed 32 KB L1 cache create a massive bandwidth bottleneck, causing the execution time to spike drastically. In contrast, the standard global memory kernel manages to scale much better for larger matrices. Although it relies entirely on the L1 cache for both matrices, it completely avoids the severe early serialization penalties inherent to the 8 KB constant cache architecture. Finally, the constant memory line abruptly stops at $N = 128$. This is because at $N = 256$ (next problem size) , the required data exceeds the absolute physical limit of 64 KB allocated for constant memory on the GPU, making execution impossible.

4 Conclusion

This lab demonstrates that uncoalesced memory access severely wastes bandwidth and that shared memory can be used to prevent this. This is achieved by loading data in a coalesced manner into shared memory, rearranging it locally, and then writing the data back to global memory in a perfectly coalesced manner. Furthermore, working with shared memory often requires specific algorithmic strategies, such as tiling. In this method, threads collaboratively gather data into a tile, synchronize, perform the computations, synchronize again, and finally write the results back to global memory. By repeating this process across all defined tiles, tiling drastically reduces the total number of memory bus accesses. Of course, when a task is inherently coalesced or when data will not be reused, utilizing shared memory introduces an unnecessary delay. This delay is caused by extra instructions and synchronization overhead, which can be entirely avoided by relying strictly on a global memory approach.

Constant memory is highly effective for streaming a single value to all threads simultaneously. For example, consider a physics engine where gravity (g) is a constant and m represents the varying masses of multiple falling objects. The L1 cache can fetch the array of masses from global memory while the constant cache simultaneously broadcasts g to calculate the weights of the falling bodies. On the other hand, constant memory is completely unfit for larger data structures due to the strict 64 KB hardware limit and the severe performance penalty of serialization when threads request different memory addresses.

Ultimately, if a kernel relies on large, fully coalesced memory accesses with minimal calculations, utilizing constant or shared memory provides no additional benefit. In such cases, relying strictly on global memory is the most efficient approach.

A Code Implementations

A.1 Part 1:

A.1.1 CUDA code

```
#include <iostream>
#include <vector>
#include <chrono>
#include <cuda_runtime.h>
#include <cstdlib>
```

```
6
7  #define TILE_DIM 32
8
9  void cpuCopy(const std::vector<float>& in, std::vector<float>& out, int N) {
10     for (int i = 0; i < N * N; ++i) {
11         out[i] = in[i];
12     }
13 }
14
15 void cpuTranspose(const std::vector<float>& in, std::vector<float>& out, int N) {
16     for (int y = 0; y < N; ++y) {
17         for (int x = 0; x < N; ++x) {
18             out[x * N + y] = in[y * N + x];
19         }
20     }
21 }
22
23 __global__ void copyGlobal(const float *d_in, float *d_out, int N) {
24     int x = blockIdx.x * blockDim.x + threadIdx.x;
25     int y = blockIdx.y * blockDim.y + threadIdx.y;
26
27     if (x < N && y < N) {
28         d_out[y * N + x] = d_in[y * N + x];
29     }
30 }
31
32 __global__ void copyShared(const float *d_in, float *d_out, int N) {
33     __shared__ float tile[TILE_DIM][TILE_DIM];
34
35     int x = blockIdx.x * TILE_DIM + threadIdx.x;
36     int y = blockIdx.y * TILE_DIM + threadIdx.y;
37
38     if (x < N && y < N) {
39         tile[threadIdx.y][threadIdx.x] = d_in[y * N + x];
40     }
41
42     __syncthreads();
43
44     if (x < N && y < N) {
45         d_out[y * N + x] = tile[threadIdx.y][threadIdx.x];
46     }
47 }
48
49 __global__ void transposeGlobal(const float *d_in, float *d_out, int N) {
50     int x = blockIdx.x * blockDim.x + threadIdx.x;
51     int y = blockIdx.y * blockDim.y + threadIdx.y;
52
53     if (x < N && y < N) {
54         d_out[x * N + y] = d_in[y * N + x];
55     }
56 }
57
58 __global__ void transposeShared(const float *d_in, float *d_out, int N) {
59     __shared__ float tile[TILE_DIM][TILE_DIM + 1];
60
61     int blockIdx_x = (blockIdx.x + blockIdx.y) % gridDim.x;
62     int blockIdx_y = blockIdx.x;
63
64     int x = blockIdx_x * TILE_DIM + threadIdx.x;
65     int y = blockIdx_y * TILE_DIM + threadIdx.y;
66
67     if (x < N && y < N) {
68         tile[threadIdx.y][threadIdx.x] = d_in[y * N + x];
69     }
70
71     __syncthreads();
72
73     x = blockIdx_y * TILE_DIM + threadIdx.x;
74     y = blockIdx_x * TILE_DIM + threadIdx.y;
75
76     if (x < N && y < N) {
77         d_out[y * N + x] = tile[threadIdx.x][threadIdx.y];
78     }
```

```
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cout << "Usage: ./session4_1 <matrix_size>\n";
        return 1;
    }

    int N = std::atoi(argv[1]);

    std::vector<float> vector_in(N * N);
    std::vector<float> vector_copy_cpu(N * N);
    std::vector<float> vector_copy_global(N * N);
    std::vector<float> vector_copy_shared(N * N);
    std::vector<float> vector_trans_cpu(N * N);
    std::vector<float> vector_trans_global(N * N);
    std::vector<float> vector_trans_shared(N * N);

    for (float i = 0; i < N * N; ++i) {
        vector_in[i] = i;
    }

    float *d_in, *d_out;
    cudaMalloc(&d_in, N * N * sizeof(float));
    cudaMalloc(&d_out, N * N * sizeof(float));
    cudaMemcpy(d_in, vector_in.data(), N * N * sizeof(float), cudaMemcpyHostToDevice);

    dim3 block(TILE_DIM, TILE_DIM);
    dim3 grid((N + TILE_DIM - 1) / TILE_DIM, (N + TILE_DIM - 1) / TILE_DIM);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    auto start_cpu_copy = std::chrono::high_resolution_clock::now();
    cpuCopy(vector_in, vector_copy_cpu, N);
    auto end_cpu_copy = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> time_cpu_copy = end_cpu_copy - start_cpu_copy;

    cudaEventRecord(start);
    copyGlobal<<<grid, block>>>(d_in, d_out, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float time_copy_global = 0;
    cudaEventElapsedTime(&time_copy_global, start, stop);
    cudaMemcpy(vector_copy_global.data(), d_out, N * N * sizeof(float),
    cudaMemcpyDeviceToHost);

    cudaEventRecord(start);
    copyShared<<<grid, block>>>(d_in, d_out, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float time_copy_shared = 0;
    cudaEventElapsedTime(&time_copy_shared, start, stop);
    cudaMemcpy(vector_copy_shared.data(), d_out, N * N * sizeof(float),
    cudaMemcpyDeviceToHost);

    auto start_cpu_trans = std::chrono::high_resolution_clock::now();
    cpuTranspose(vector_in, vector_trans_cpu, N);
    auto end_cpu_trans = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> time_cpu_trans = end_cpu_trans -
    start_cpu_trans;

    cudaEventRecord(start);
    transposeGlobal<<<grid, block>>>(d_in, d_out, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float time_trans_global = 0;
    cudaEventElapsedTime(&time_trans_global, start, stop);
    cudaMemcpy(vector_trans_global.data(), d_out, N * N * sizeof(float),
    cudaMemcpyDeviceToHost);

    cudaEventRecord(start);
```

```
    transposeShared<<<grid, block>>>(d_in, d_out, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float time_trans_shared = 0;
    cudaEventElapsedTime(&time_trans_shared, start, stop);
    cudaMemcpy(vector_trans_shared.data(), d_out, N * N * sizeof(float),
    cudaMemcpyDeviceToHost);

    float total_bytes = 2.0f * N * N * sizeof(float);
    float bw_copy_global = (total_bytes / (time_copy_global * 1e-3f)) / 1e9f;
    float bw_copy_shared = (total_bytes / (time_copy_shared * 1e-3f)) / 1e9f;
    float bw_trans_global = (total_bytes / (time_trans_global * 1e-3f)) / 1e9f;
    float bw_trans_shared = (total_bytes / (time_trans_shared * 1e-3f)) / 1e9f;

    std::cout << "--- COPY KERNEL ---\n";
    std::cout << "CPU Time: " << time_cpu_copy.count() << " ms\n";
    std::cout << "GPU Global Time: " << time_copy_global << " ms\n";
    std::cout << "GPU Global BW: " << bw_copy_global << " GB/s\n";
    std::cout << "GPU Shared Time: " << time_copy_shared << " ms\n";
    std::cout << "GPU Shared BW: " << bw_copy_shared << " GB/s\n";

    std::cout << "\n--- TRANSPOSE KERNEL ---\n";
    std::cout << "CPU Time: " << time_cpu_trans.count() << " ms\n";
    std::cout << "GPU Global Time: " << time_trans_global << " ms\n";
    std::cout << "GPU Global BW: " << bw_trans_global << " GB/s\n";
    std::cout << "GPU Shared Time: " << time_trans_shared << " ms\n";
    std::cout << "GPU Shared BW: " << bw_trans_shared << " GB/s\n";

    bool match_copy_global = true;
    bool match_copy_shared = true;
    bool match_trans_global = true;
    bool match_trans_shared = true;

    for (int i = 0; i < N * N; ++i) {
        if (vector_copy_cpu[i] != vector_copy_global[i]) match_copy_global = false;
        if (vector_copy_cpu[i] != vector_copy_shared[i]) match_copy_shared = false;
        if (vector_trans_cpu[i] != vector_trans_global[i]) match_trans_global = false;
        if (vector_trans_cpu[i] != vector_trans_shared[i]) match_trans_shared = false;
    }

    std::cout << "\n--- VALIDATION ---\n";
    std::cout << "Copy Global Match: " << (match_copy_global ? "Yes" : "No") << "\n";
    std::cout << "Copy Shared Match: " << (match_copy_shared ? "Yes" : "No") << "\n";
    std::cout << "Transpose Global Match: " << (match_trans_global ? "Yes" : "No") << "\n";
    std::cout << "Transpose Shared Match: " << (match_trans_shared ? "Yes" : "No") << "\n";

    cudaFree(d_in);
    cudaFree(d_out);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return 0;
}
```

A.1.2 Data Collection Script

```
import subprocess
import csv
import re
import sys
import matplotlib.pyplot as plt

matrix_sizes = [128, 256, 512, 1024, 2048, 4096, 8192]
runs_per_size = 20
csv_file = "results_session4_1.csv"

results = []

for n in matrix_sizes:
    print(f"Running N = {n:>4} ...", end=" ", flush=True)

    copy_cpu_t_tot = 0.0
    copy_glob_t_tot = 0.0
```

```
    copy_glob_bw_tot = 0.0
    copy_shar_t_tot = 0.0
    copy_shar_bw_tot = 0.0

    trans_cpu_t_tot = 0.0
    trans_glob_t_tot = 0.0
    trans_glob_bw_tot = 0.0
    trans_shar_t_tot = 0.0
    trans_shar_bw_tot = 0.0

    valid_runs = 0

    for _ in range(runs_per_size):
        result = subprocess.run(
            ["./session4_1", str(n)],
            capture_output=True,
            text=True
        )

        output = result.stdout

        cpu_t_matches = re.findall(r"CPU Time:\s*([0-9a-zA-Z.+-]+)", output)
        glob_t_matches = re.findall(r"GPU Global Time:\s*([0-9a-zA-Z.+-]+)", output)
        glob_bw_matches = re.findall(r"GPU Global BW:\s*([0-9a-zA-Z.+-]+)", output)
        shar_t_matches = re.findall(r"GPU Shared Time:\s*([0-9a-zA-Z.+-]+)", output)
        shar_bw_matches = re.findall(r"GPU Shared BW:\s*([0-9a-zA-Z.+-]+)", output)

        if (len(cpu_t_matches) == 2 and len(glob_t_matches) == 2 and
            len(glob_bw_matches) == 2 and len(shar_t_matches) == 2 and
            len(shar_bw_matches) == 2):

            try:
                copy_cpu_t_tot += float(cpu_t_matches[0])
                copy_glob_t_tot += float(glob_t_matches[0])
                copy_glob_bw_tot += float(glob_bw_matches[0])
                copy_shar_t_tot += float(shar_t_matches[0])
                copy_shar_bw_tot += float(shar_bw_matches[0])

                trans_cpu_t_tot += float(cpu_t_matches[1])
                trans_glob_t_tot += float(glob_t_matches[1])
                trans_glob_bw_tot += float(glob_bw_matches[1])
                trans_shar_t_tot += float(shar_t_matches[1])
                trans_shar_bw_tot += float(shar_bw_matches[1])

                valid_runs += 1
            except ValueError:
                pass
        else:
            if _ == 0:
                print(f"\nParse Fout! Output van C++ was:\n{output}")

    if valid_runs > 0:
        results.append((
            n,
            copy_cpu_t_tot / valid_runs,
            copy_glob_t_tot / valid_runs,
            copy_shar_t_tot / valid_runs,
            copy_glob_bw_tot / valid_runs,
            copy_shar_bw_tot / valid_runs,
            trans_cpu_t_tot / valid_runs,
            trans_glob_t_tot / valid_runs,
            trans_shar_t_tot / valid_runs,
            trans_glob_bw_tot / valid_runs,
            trans_shar_bw_tot / valid_runs
        ))
        print(f"Done ({valid_runs}/{runs_per_size})")
    else:
        print("Failed")

if not results:
    sys.exit()

with open(csv_file, mode='w', newline='') as f:
```

```
    writer = csv.writer(f)
    writer.writerow([
        "N", "Copy_CPU_ms", "Copy_Global_ms", "Copy_Shared_ms", "Copy_Global_GBs", "Copy_Shared_GBs",
        "Trans_CPU_ms", "Trans_Global_ms", "Trans_Shared_ms", "Trans_Global_GBs", "Trans_Shared_GBs"
    ])
    for row in results:
        writer.writerow(row)

x_sizes = [r[0] for r in results]
y_copy_cpu_t = [r[1] for r in results]
y_copy_glob_t = [r[2] for r in results]
y_copy_shar_t = [r[3] for r in results]
y_copy_glob_bw = [r[4] for r in results]
y_copy_shar_bw = [r[5] for r in results]

y_trans_cpu_t = [r[6] for r in results]
y_trans_glob_t = [r[7] for r in results]
y_trans_shar_t = [r[8] for r in results]
y_trans_glob_bw = [r[9] for r in results]
y_trans_shar_bw = [r[10] for r in results]

fig, axs = plt.subplots(2, 2, figsize=(16, 12))

axs[0, 0].plot(x_sizes, y_copy_cpu_t, marker='o', label="CPU Time")
axs[0, 0].plot(x_sizes, y_copy_glob_t, marker='s', label="GPU Global Time")
axs[0, 0].plot(x_sizes, y_copy_shar_t, marker='^', label="GPU Shared Time")
axs[0, 0].set_xlabel("Matrix Size (N)")
axs[0, 0].set_ylabel("Time (ms)")
axs[0, 0].set_title("Copy Kernel Time")
axs[0, 0].set_xscale("log", base=2)
axs[0, 0].set_yscale("log")
axs[0, 0].grid(True, which="both", ls="--", lw=0.5)
axs[0, 0].legend()

axs[0, 1].plot(x_sizes, y_trans_cpu_t, marker='o', label="CPU Time")
axs[0, 1].plot(x_sizes, y_trans_glob_t, marker='s', label="GPU Global Time")
axs[0, 1].plot(x_sizes, y_trans_shar_t, marker='^', label="GPU Shared Time")
axs[0, 1].set_xlabel("Matrix Size (N)")
axs[0, 1].set_ylabel("Time (ms)")
axs[0, 1].set_title("Transpose Kernel Time")
axs[0, 1].set_xscale("log", base=2)
axs[0, 1].set_yscale("log")
axs[0, 1].grid(True, which="both", ls="--", lw=0.5)
axs[0, 1].legend()

axs[1, 0].plot(x_sizes, y_copy_glob_bw, marker='s', label="GPU Global BW")
axs[1, 0].plot(x_sizes, y_copy_shar_bw, marker='^', label="GPU Shared BW")
axs[1, 0].set_xlabel("Matrix Size (N)")
axs[1, 0].set_ylabel("Bandwidth (GB/s)")
axs[1, 0].set_title("Copy Kernel Bandwidth")
axs[1, 0].set_xscale("log", base=2)
axs[1, 0].grid(True, which="both", ls="--", lw=0.5)
axs[1, 0].legend()

axs[1, 1].plot(x_sizes, y_trans_glob_bw, marker='s', label="GPU Global BW")
axs[1, 1].plot(x_sizes, y_trans_shar_bw, marker='^', label="GPU Shared BW")
axs[1, 1].set_xlabel("Matrix Size (N)")
axs[1, 1].set_ylabel("Bandwidth (GB/s)")
axs[1, 1].set_title("Transpose Kernel Bandwidth")
axs[1, 1].set_xscale("log", base=2)
axs[1, 1].grid(True, which="both", ls="--", lw=0.5)
axs[1, 1].legend()

plt.tight_layout()
plt.show()
```

A.2 Part 2:

A.2.1 CUDA code

```
#include <iostream>
```

```cpp
#include <vector>
#include <chrono>
#include <cuda_runtime.h>
#include <cstdlib>

#define TILE_DIM 32
#define MAX_CONST_SIZE 128

__constant__ float d_a_const[MAX_CONST_SIZE * MAX_CONST_SIZE];

void multiplyCPU(const std::vector<float>& A, const std::vector<float>& B, std::vector<float>& C, int N) {
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            float s = 0;
            for (int k = 0; k < N; ++k) s += A[i * N + k] * B[k * N + j];
            C[i * N + j] = s;
        }
    }
}

__global__ void multiplyGlobal(const float* A, const float* B, float* C, int N) {
    int c = blockIdx.x * blockDim.x + threadIdx.x;
    int r = blockIdx.y * blockDim.y + threadIdx.y;
    if (r < N && c < N) {
        float s = 0;
        for (int k = 0; k < N; ++k) s += A[r * N + k] * B[k * N + c];
        C[r * N + c] = s;
    }
}

__global__ void multiplyShared(const float* A, const float* B, float* C, int N) {
    __shared__ float tA[TILE_DIM][TILE_DIM];
    __shared__ float tB[TILE_DIM][TILE_DIM];

    int r = blockIdx.y * TILE_DIM + threadIdx.y;
    int c = blockIdx.x * TILE_DIM + threadIdx.x;
    float s = 0;

    for (int m = 0; m < (N + TILE_DIM - 1) / TILE_DIM; ++m) {
        if (r < N && m * TILE_DIM + threadIdx.x < N)
            tA[threadIdx.y][threadIdx.x] = A[r * N + m * TILE_DIM + threadIdx.x];
        else
            tA[threadIdx.y][threadIdx.x] = 0;

        if (m * TILE_DIM + threadIdx.y < N && c < N)
            tB[threadIdx.y][threadIdx.x] = B[(m * TILE_DIM + threadIdx.y) * N + c];
        else
            tB[threadIdx.y][threadIdx.x] = 0;

        __syncthreads();
        for (int k = 0; k < TILE_DIM; ++k) s += tA[threadIdx.y][k] * tB[k][threadIdx.x];
        __syncthreads();
    }
    if (r < N && c < N) C[r * N + c] = s;
}

__global__ void multiplyConstant(const float* B, float* C, int N) {
    int c = blockIdx.x * blockDim.x + threadIdx.x;
    int r = blockIdx.y * blockDim.y + threadIdx.y;
    if (r < N && c < N) {
        float s = 0;
        for (int k = 0; k < N; ++k) {
            s += d_a_const[r * N + k] * B[k * N + c];
        }
        C[r * N + c] = s;
    }
}

int main(int argc, char* argv[]) {
    if (argc != 2) return 1;
    int N = std::atoi(argv[1]);

```

```
    std::vector<float> h_A(N * N, 1.0f), h_B(N * N, 2.0f), h_C(N * N);
    std::vector<float> h_C_GPU(N * N, 0.0f);

    auto start = std::chrono::high_resolution_clock::now();
    multiplyCPU(h_A, h_B, h_C, N);
    auto end = std::chrono::high_resolution_clock::now();
    float cpu_time = std::chrono::duration<double, std::milli>(end - start).count();

    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, N * N * sizeof(float));
    cudaMalloc(&d_B, N * N * sizeof(float));
    cudaMalloc(&d_C, N * N * sizeof(float));

    cudaEvent_t evS, evE;
    cudaEventCreate(&evS);
    cudaEventCreate(&evE);

    cudaEventRecord(evS);
    cudaMemcpy(d_A, h_A.data(), N * N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), N * N * sizeof(float), cudaMemcpyHostToDevice);
    if (N <= MAX_CONST_SIZE) {
        cudaMemcpyToSymbol(d_a_const, h_A.data(), N * N * sizeof(float));
    }
    cudaEventRecord(evE);
    cudaEventSynchronize(evE);
    float t_mig; cudaEventElapsedTime(&t_mig, evS, evE);

    dim3 bk(TILE_DIM, TILE_DIM);
    dim3 gd((N + TILE_DIM - 1) / TILE_DIM, (N + TILE_DIM - 1) / TILE_DIM);

    cudaEventRecord(evS);
    multiplyGlobal<<<gd, bk>>>(d_A, d_B, d_C, N);
    cudaEventRecord(evE);
    cudaEventSynchronize(evE);
    float t_glo; cudaEventElapsedTime(&t_glo, evS, evE);

    // Verificatie van Global Kernel resultaat
    cudaMemcpy(h_C_GPU.data(), d_C, N * N * sizeof(float), cudaMemcpyDeviceToHost);
    bool match = true;
    for (int i = 0; i < N * N; ++i) {
        if (std::abs(h_C[i] - h_C_GPU[i]) > 1e-4) {
            match = false; break;
        }
    }

    cudaEventRecord(evS);
    multiplyShared<<<gd, bk>>>(d_A, d_B, d_C, N);
    cudaEventRecord(evE);
    cudaEventSynchronize(evE);
    float t_sha; cudaEventElapsedTime(&t_sha, evS, evE);

    float t_con = 0;
    if (N <= MAX_CONST_SIZE) {
        cudaEventRecord(evS);
        multiplyConstant<<<gd, bk>>>(d_B, d_C, N);
        cudaEventRecord(evE);
        cudaEventSynchronize(evE);
        cudaEventElapsedTime(&t_con, evS, evE);
    }

    std::cout << "MIG: " << t_mig << "\n";
    std::cout << "CPU: " << cpu_time << "\n";
    std::cout << "GLO: " << t_glo << "\n";
    std::cout << "SHA: " << t_sha << "\n";
    std::cout << "CON: " << (N <= MAX_CONST_SIZE ? std::to_string(t_con) : "N/A") << "\n";
    std::cout << "VER: " << (match ? "Pass" : "Fail") << "\n";

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    cudaEventDestroy(evS); cudaEventDestroy(evE);
    return 0;
}
```

A.2.2 Data Collection Scripts

```python
import subprocess
import csv
import re
import sys
import matplotlib.pyplot as plt

matrix_sizes = [32, 64, 128, 256, 512, 1024, 2048]
runs = 5
csv_file = "results_session4_2_time.csv"
results = []

for n in matrix_sizes:
    print(f"Testing N = {n:>4} ({runs}x)...", end=" ", flush=True)

    t_mig, t_cpu, t_glo, t_sha, t_con, count = 0.0, 0.0, 0.0, 0.0, 0.0, 0
    verification_passed = True

    for _ in range(runs):
        process = subprocess.run(["./session4_2", str(n)], capture_output=True, text=True)
        out = process.stdout

        m_mig = re.search(r"MIG:\s*([\d.]+)", out)
        m_cpu = re.search(r"CPU:\s*([\d.]+)", out)
        m_glo = re.search(r"GLO:\s*([\d.]+)", out)
        m_sha = re.search(r"SHA:\s*([\d.]+)", out)
        m_con = re.search(r"CON:\s*(N/A|[\d.]+)", out)
        m_ver = re.search(r"VER:\s*(Pass|Fail)", out)

        if m_ver and m_ver.group(1) == "Fail":
            verification_passed = False

        if m_mig and m_cpu and m_glo and m_sha and m_con:
            t_mig += float(m_mig.group(1))
            t_cpu += float(m_cpu.group(1))
            t_glo += float(m_glo.group(1))
            t_sha += float(m_sha.group(1))
            if m_con.group(1) != "N/A":
                t_con += float(m_con.group(1))
            count += 1

    if not verification_passed:
        print("Failed (Verification CPU vs GPU mismatch)")
    elif count > 0:
        results.append((
            n,
            t_mig/count,
            t_cpu/count,
            t_glo/count,
            t_sha/count,
            (t_con/count if n <= 128 else None)
        ))
        print("Done")
    else:
        print("Failed (Parse Error)")

if not results:
    sys.exit()

with open(csv_file, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["N", "Migration_ms", "CPU_ms", "Global_ms", "Shared_ms", "Constant_ms"])
    for row in results:
        csv_row = list(row)
        if csv_row[5] is None:
            csv_row[5] = "N/A"
        writer.writerow(csv_row)

x_sizes = [r[0] for r in results]
y_mig = [r[1] for r in results]
y_cpu = [r[2] for r in results]
y_glo = [r[3] for r in results]
```

```
y_sha = [r[4] for r in results]

x_con = [r[0] for r in results if r[5] is not None]
y_con = [r[5] for r in results if r[5] is not None]

plt.figure(figsize=(10, 6))

plt.plot(x_sizes, y_cpu, marker='o', label="CPU Time")
plt.plot(x_sizes, y_glo, marker='s', label="GPU Global Time")
plt.plot(x_sizes, y_sha, marker='^', label="GPU Shared Time")
if x_con:
    plt.plot(x_con, y_con, marker='d', label="GPU Constant Time", color='red', linestyle='--'
    )

plt.xlabel("Matrix Size (N)")
plt.ylabel("Execution Time (ms)")
plt.title("Execution Times: Global, Shared and Constant Memory")
plt.xscale("log", base=2)
plt.yscale("log")
plt.grid(True, which="both", ls="--", lw=0.5)
plt.legend()

plt.tight_layout()
plt.savefig("plot4_2_times_only.png")
plt.show()
```

References

[1] NVIDIA Corporation. *CUDA C++ Programming Guide*. This was used as a basis to calculate the values of the table.

[2] TechPowerUp. *NVIDIA T400 4 GB Specs - GPU Database*. https://www.techpowerup.com/gpu-specs/t400-4-gb.c3843